SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3) **Total Marks: 40**

Question Count: 5

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: beffy

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

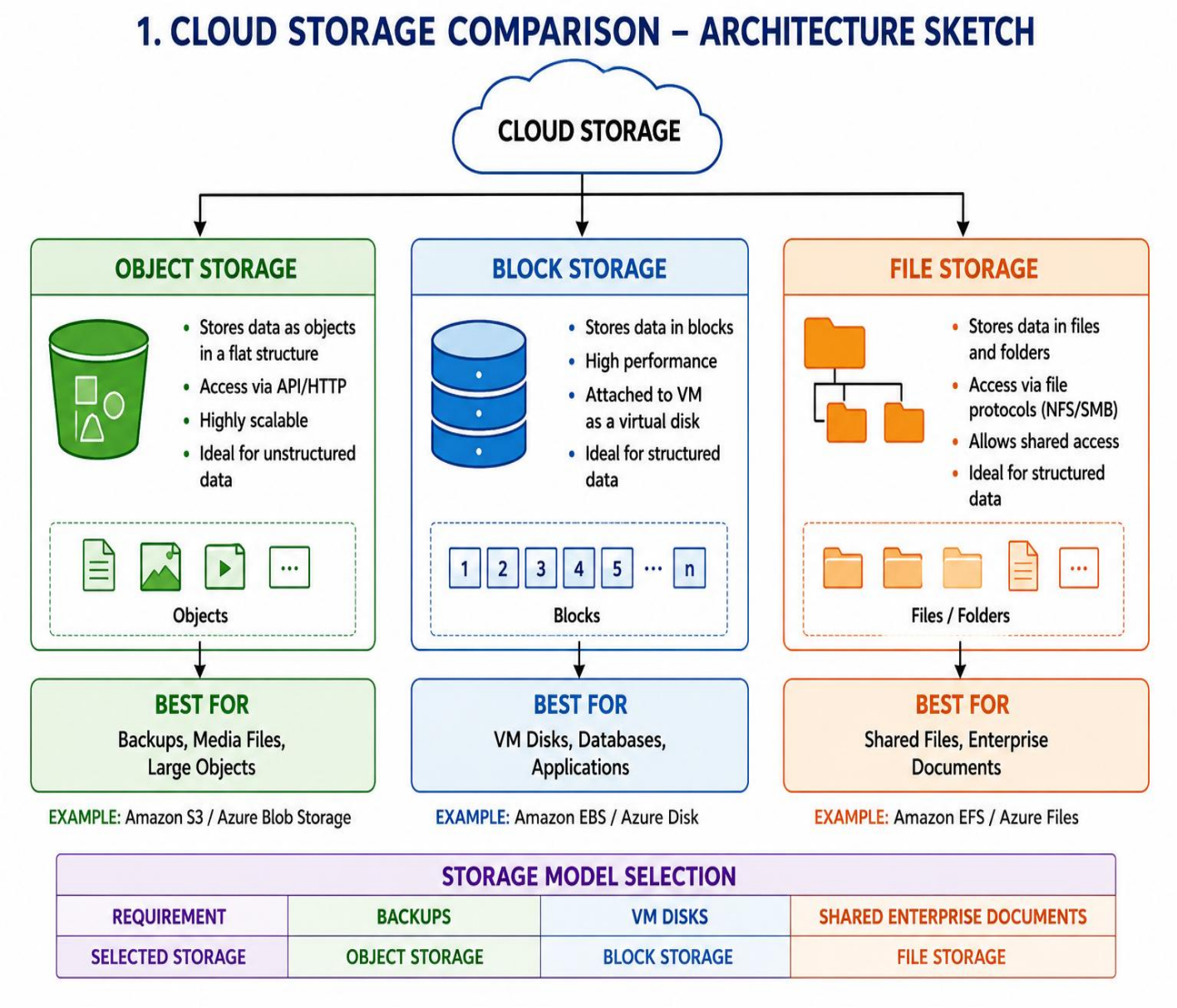
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

EXPERIMENT 1: Cloud Storage Comparison

Objective:

To compare Object Storage, Block Storage, and File Storage and select the most suitable storage model for backups, VM disks, and shared enterprise documents.

Architecture Sketch:



Comparison:

Feature	Object Storage	Block Storage	File Storage
Data type	Objects/files	Blocks	Files/folders
Access	API/HTTP	Attached to VM	File system
Scalability	Very high	High	High
Best for	Backups	VM disks	Shared documents
Example	Amazon S3	Amazon EBS	Amazon EFS

Procedure:

1. Identify the three major cloud storage models.
2. Study how each storage model stores and accesses data.
3. Compare scalability, performance, accessibility and cost.
4. Map each storage model to a practical cloud requirement.
5. Select the appropriate storage model for each application.
6. Document the comparison.

Selection

- **Backups → Object Storage**
    - Suitable for large amounts of backup data.
    - Highly scalable.
    - Provides durable storage.

- **VM Disks → Block Storage**
    - Provides high-performance disk-like storage.
    - Can be attached to virtual machines.
    - Suitable for operating systems and databases.
- **Shared Enterprise Documents → File Storage**
    - Supports folders and file-based access.
    - Multiple users and systems can access shared files.

Expected Output:

Requirement	Selected Storage
Backup	Object Storage
VM Disk	Block Storage
Enterprise Documents	File Storage

Conclusion:

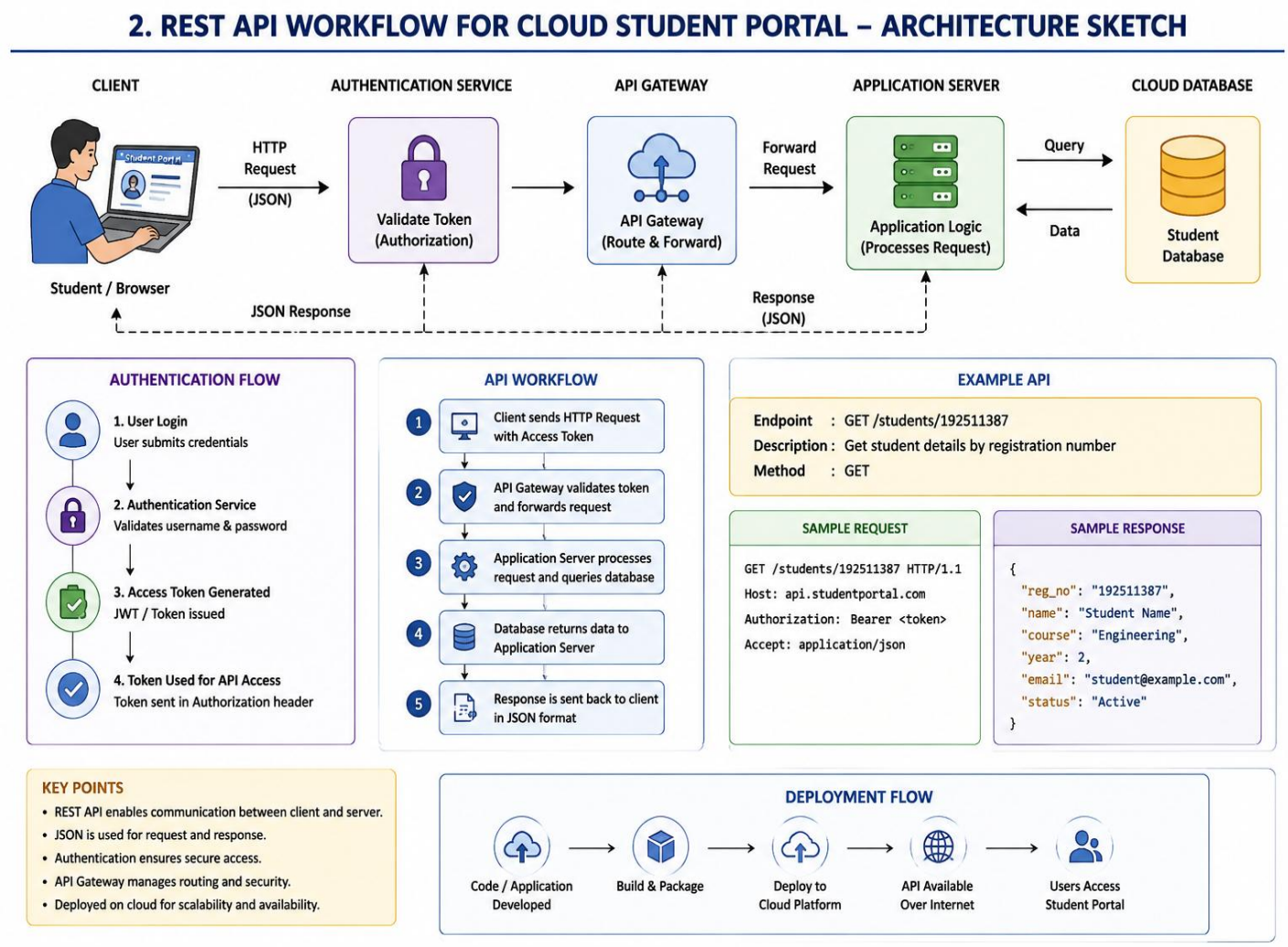
Object storage is best for backups, block storage is best for VM disks, and file storage is best for shared enterprise documents. Each model is selected according to its access method and application requirements.

EXPERIMENT 2: REST API Workflow for Cloud Student Portal

Objective:

To develop and demonstrate a simple REST API workflow for a cloud-based student portal including request, response, authentication and deployment.

Architecture Sketch:



Example API:

GET /students/192511387

Request

GET /students/192511387

Authorization: Bearer <token>

Response


```
{
  "reg_no": "192511387",
  "name": "Student",
  "course": "Engineering",
  "year": 2
}
```


Procedure

1. Create a simple cloud-based student portal.
2. Create REST API endpoints for student information.
3. Use HTTP methods such as GET, POST, PUT and DELETE.
4. Implement authentication using an access token.
5. Send requests from a browser or API testing tool.
6. Validate the request at the API server.
7. Retrieve required information from the cloud database.
8. Return the result as a JSON response.
9. Deploy the API on a cloud platform.
10. Test the deployed API.

Authentication Flow

Student

|

| Login

v

Authentication Service

|

| Access Token

v

Student Portal

|

| Token + API Request

v

REST API

|

v

Database

Expected Output:

Request → GET /students/192511387

↓

Authentication Verified

↓

Cloud Database

↓

JSON Response

↓

Student Details Displayed

Conclusion:

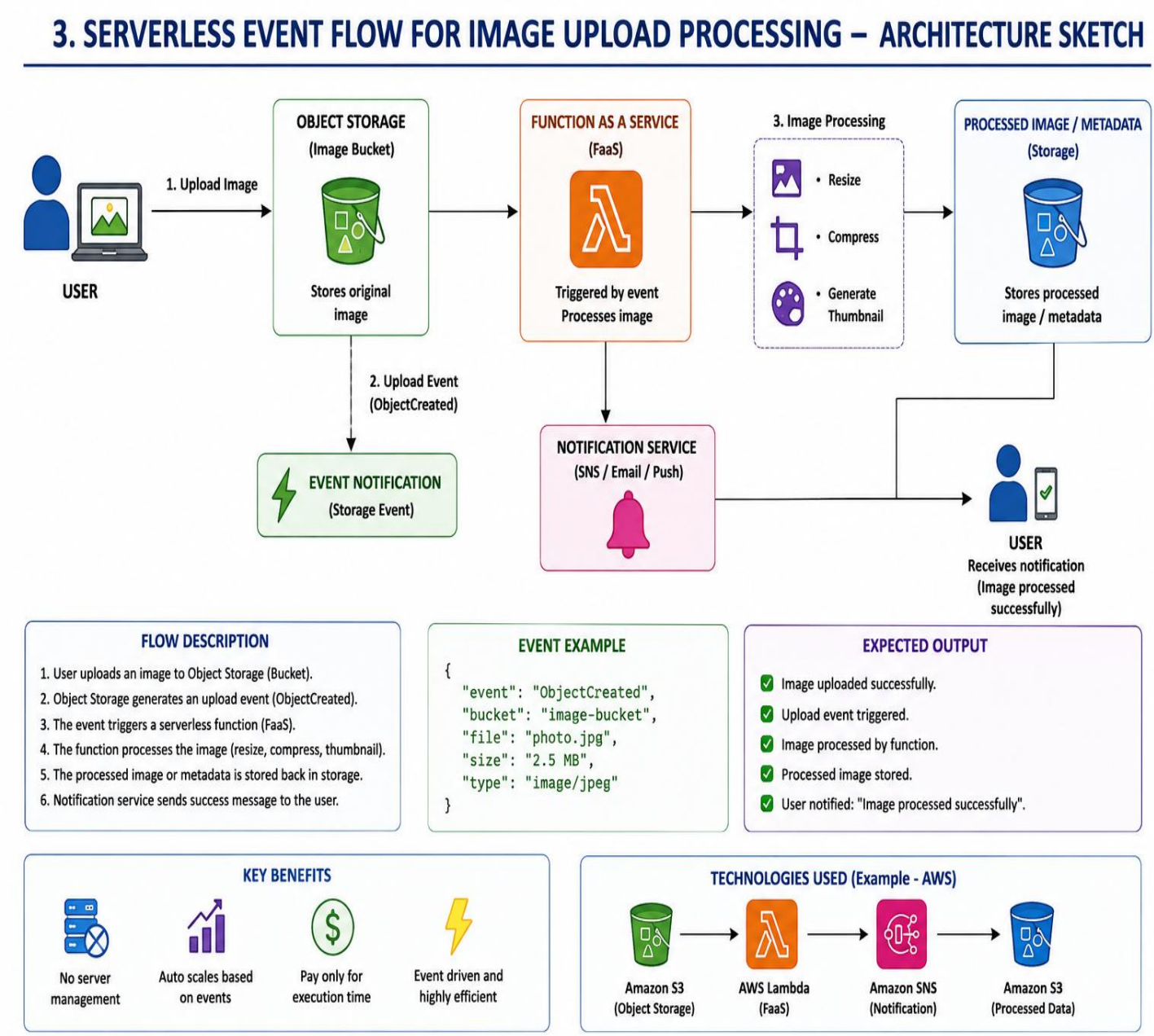
A REST API provides communication between the student portal, cloud application and database. Authentication ensures that only authorized users can access student information.

EXPERIMENT 3: Serverless Image Upload Processing

Objective

To design a **serverless event-driven architecture** for processing images uploaded to cloud object storage using Function-as-a-Service and a notification service.

Architecture Sketch



Procedure

1. Create a cloud object storage bucket.
2. Configure the bucket to accept image uploads.
3. Create a serverless function.
4. Configure the object storage service to generate an event when an image is uploaded.
5. Connect the upload event to the serverless function.
6. The function receives information about the uploaded image.
7. The function processes the image.
8. Store the processed image or metadata.
9. Trigger the notification service.
10. Send a notification to the user after successful processing.

Example Event

```
{
  "event": "ObjectCreated",
  "file": "student_photo.jpg",
  "type": "image/jpeg"
}
```

Processing Flow

Upload Image

↓

Object Storage

↓

Upload Event

↓

Serverless Function

↓

Image Processing

↓

Notification Service

↓

"Image Processed Successfully"

Expected Output

Image: student_photo.jpg

Status: Uploaded

Processing: Completed

Notification: Sent Successfully

Advantages

- No dedicated server required.
- Function runs only when an event occurs.
- Automatically scales according to workload.
- Suitable for event-driven applications.

Conclusion

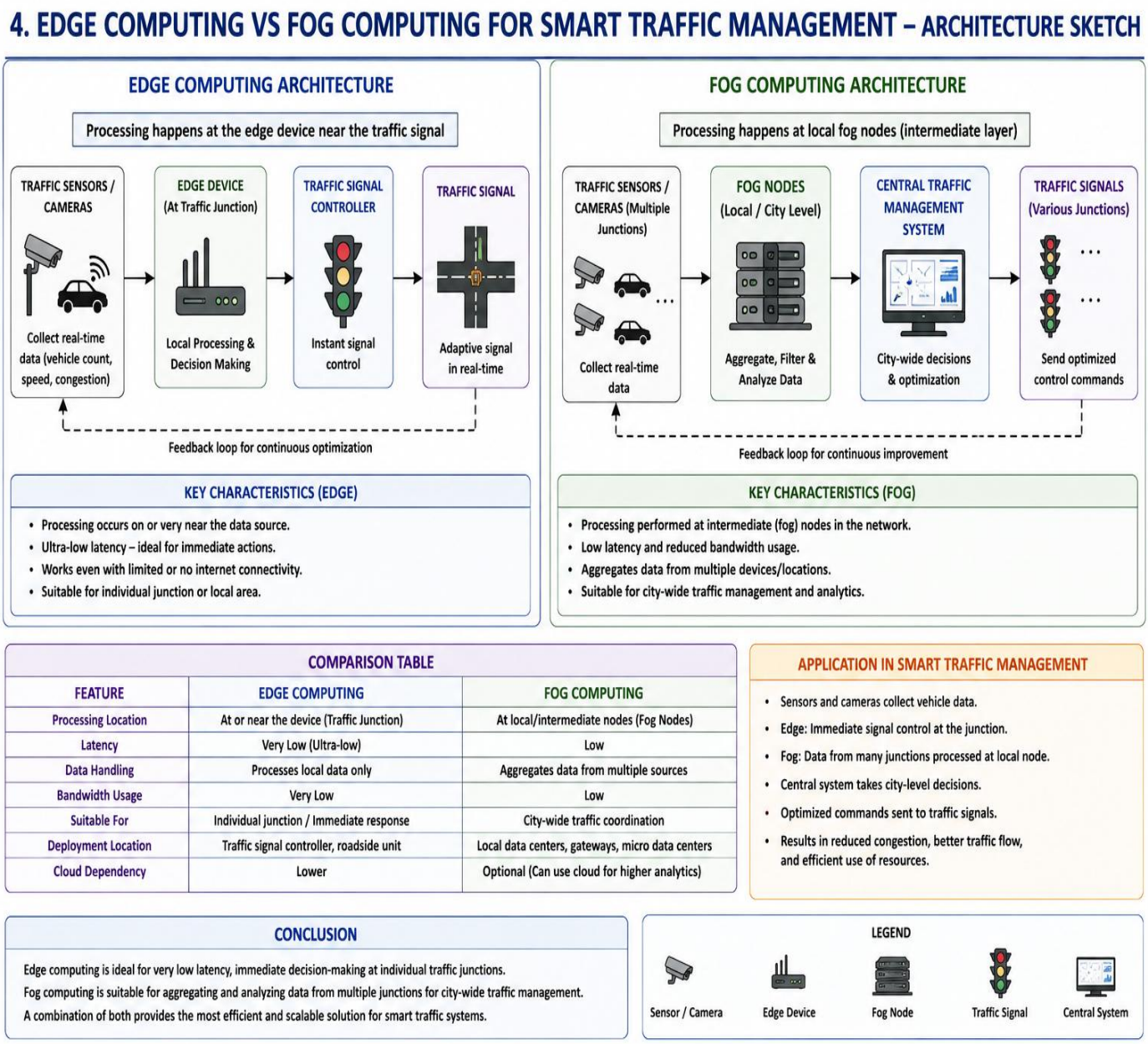
The serverless architecture automatically processes images whenever they are uploaded. Object storage handles the image, FaaS performs the processing, and the notification service informs the user.

EXPERIMENT 4: Edge Computing vs Fog Computing for Smart Traffic Management

Objective

To compare Edge Computing and Fog Computing for a smart traffic management system based on architecture, latency and deployment location.

Architecture



Comparison

Feature	Edge Computing	Fog Computing
Processing location	Directly near device	Intermediate/local network
Latency	Very low	Low
Bandwidth usage	Reduced	Reduced
Suitable for	Individual junction	City-wide traffic
Architecture	More distributed	Hierarchical/distributed
Cloud dependency	Lower	Can use cloud

Procedure

1. Install traffic cameras and sensors at road junctions.
2. Collect vehicle count, speed and congestion data.
3. For edge computing, process the data directly at the local edge device.
4. For fog computing, send data to nearby fog nodes.
5. Analyze traffic conditions.
6. Generate traffic-control decisions.
7. Send commands to traffic signals.
8. Compare response time and deployment locations.
9. Determine which architecture is more suitable.

Latency Consideration

Edge:

Sensor → Edge Device → Traffic Signal

↓

Very Low Latency

Fog:

Sensor → Fog Node → Traffic Signal

↓

Low Latency

↓

Cloud

Deployment Location

Edge:

- Traffic signal controller
- Roadside gateway
- Local traffic camera device

Fog:

- Traffic control center
- Local network gateway
- City-level computing node

Advantages of Edge and Fog Computing

Edge Computing

- Provides very fast response for real-time traffic decisions.
- Reduces the amount of data sent to the cloud.
- Continues basic processing even with limited connectivity.

Fog Computing

- Combines data from multiple traffic junctions.
- Supports city-wide traffic analysis and coordination.
- Reduces cloud workload by processing data locally.

Expected Output

Requirement	**Better Choice**
Immediate traffic signal response	Edge
Individual junction processing	Edge
City-wide traffic coordination	Fog
Local aggregation of sensor data	Fog

Conclusion

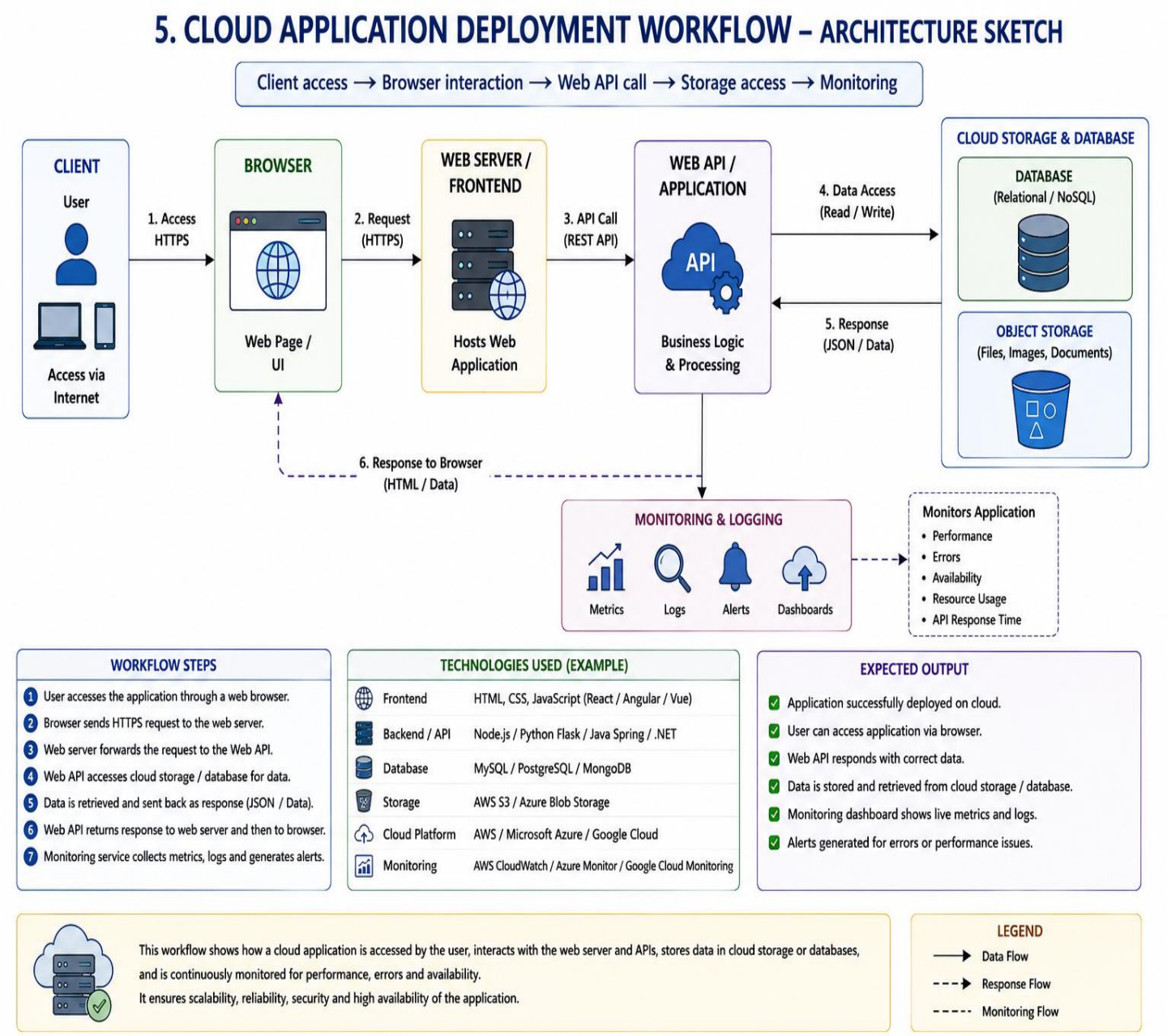
Edge computing is suitable when very low latency and immediate decisions are required at individual traffic locations. Fog computing is useful when data from multiple traffic devices needs to be aggregated and processed at an intermediate level.

EXPERIMENT 5: Cloud Application Deployment Workflow

Objective

To demonstrate a complete cloud application deployment workflow involving client access, browser interaction, web API calls, storage access and monitoring.

Architecture Sketch



Procedure

1. Develop a simple cloud application.
2. Create the application frontend.
3. Deploy the application to a cloud platform.
4. Access the application using a web browser.
5. The browser sends requests to the web server.
6. The web application calls the required web APIs.
7. APIs communicate with cloud storage/database.
8. Data is returned to the application.
9. The browser displays the result to the user.
10. Configure monitoring and logging to observe application performance.

Complete Workflow

Client

↓

Browser

↓

Web Application

↓

Web API

↓

Cloud Storage / Database

↓

API Response

↓

Web Application

↓

Browser

↓

User

Example

User opens:

https://cloudstudentportal.com

↓

Browser sends HTTPS request

↓

Web server receives request

↓

API requests student data

↓

Cloud database returns data

↓

Web application processes response

↓

Student information displayed

Monitoring

The cloud monitoring service can be used to observe:

- Application availability
- CPU and memory usage
- API response time
- Errors and failures
- Request traffic
- Storage usage

Expected Output

Client Access → Successful

Browser Interaction → Successful

Web API Call → Successful

Storage Access → Successful

Application Response → Displayed

Monitoring → Active

Conclusion

The cloud deployment workflow connects the client, browser, web application, APIs, cloud storage and monitoring services. This provides a complete cloud-based application environment that can be accessed through the internet.